

10. Implement collision between any player character and the game world.

STEP 1 — Create a New Unity Project

1. Open Unity Hub

2. Click:

New Project

3. Select:

3D Core

4. Give project name:

CollisionAssignment

5. Click:

Create Project

STEP 2 — Create the Ground

1. In Hierarchy:

Right Click → 3D Object → Plane

Rename it:

Ground

This acts as the floor.

STEP 3 — Create the Player

1. In Hierarchy:

Right Click → 3D Object → Capsule

Rename it:

Player

STEP 4 — Add Rigidbody to Player

1. Select Player

2. In Inspector click:

Add Component

3. Search:

Rigidbody

4. Add it.

STEP 5 — Prevent Player From Falling

In Rigidbody:

Uncheck:

Use Gravity

Freeze unnecessary rotations:

Constraints

Check:

Freeze Rotation X

Freeze Rotation Z

STEP 6 — Create Walls/Obstacles

Create Cube

Hierarchy →

Right Click → 3D Object → Cube

FrontWall

Select `FrontWall` and set:

Position

```
X = 0
Y = 0.5
Z = 5
```

Scale

```
X = 10
Y = 1
Z = 1
```

BackWall

Select `BackWall` and set:

Position

```
X = 0
Y = 0.5
Z = -5
```

Scale

```
X = 10
Y = 1
Z = 1
```

Also Correct Left and Right Walls

LeftWall

Position

X = -5
Y = 0.5
Z = 0

Scale

X = 1
Y = 1
Z = 10

RightWall

Position

X = 5
Y = 0.5
Z = 0

Scale

X = 1
Y = 1
Z = 10

Ground Recommended Scale

Select Ground.

Set:

Scale X = 1
Scale Y = 1
Scale Z = 1

(Unity Plane is already large.)

STEP 7 — Create Scripts Folder

In Project window:

Right Click:

Assets → Create → Folder

Name:

Scripts

STEP 8 — Create PlayerController Script

Inside Scripts folder:

Right Click:

Create → C# Script

Name:

PlayerController

STEP 9 — Add Movement Code

Double-click script and replace everything

```
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    public float speed = 5f;
    private Rigidbody rb;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    void FixedUpdate()
    {
        float moveHorizontal = Input.GetAxisRaw("Horizontal");
        float moveVertical = Input.GetAxisRaw("Vertical");

        Vector3 movement = new Vector3(moveHorizontal, 0.0f,
moveVertical).normalized;

        rb.MovePosition(rb.position + movement * speed * Time.fixedDeltaTime);
    }
}
```

STEP 10 — Attach Script to Player

Drag:

PlayerController

onto the Player object.

OR

Select Player →

Add Component → PlayerController

STEP 11 — Create Collision Script

Inside Scripts folder:

Create new C# script:

OnCollision

Replace code with:

```
using UnityEngine;

public class OnCollision : MonoBehaviour
{
    void OnCollisionEnter(Collision collision)
    {
        if (collision.gameObject.name == "RightWall")
        {
            Debug.Log("Collided with Right Wall");
        }
        else if (collision.gameObject.name == "LeftWall")
        {
            Debug.Log("Collided with Left Wall");
        }
        else if (collision.gameObject.name == "FrontWall")
        {
            Debug.Log("Collided with Front Wall");
        }
        else if (collision.gameObject.name == "BackWall")
        {
            Debug.Log("Collided with Back Wall");
        }
    }
}
```

TEP 12 — Attach Collision Script

Drag:

OnCollision

onto the Player object.

STEP 13 — Enable Old Input System (IMPORTANT)

If you get input errors:

Go to:

Edit → Project Settings → Player

Find:

Active Input Handling

Select:

Both

Click Apply.

Restart Unity if asked.

STEP 14 — Position Camera

Select Main Camera.

Set Position:

X = 0
Y = 8
Z = -10

Set Rotation:

X = 30
Y = 0
Z = 0

STEP 15 — Test the Project

Click:

Play

Use:

W A S D

or Arrow Keys.

Move into the wall.

STEP 16 — Check Collision Message

Open Console:

Window → General → Console

You should see:

Collided with: Wall

STEP 17 — Save Scene

File →

Save As

Name:

CollisionScene

Final Output of Assignment ✓

Your assignment now includes:

- Player character
- Rigidbody physics
- Movement
- World obstacles
- Collision implementation
- Collision detection message

This fully satisfies:

Implement collision between any player character and the game world